

COMPLETE PROJECT GUIDE

AuraMatch AI

Olfactory Matching & Fragrance Recommendation Platform
— Setup, Architecture & User Guide

Semantic search over 40,000+ fragrances • FastAPI + Next.js + PostgreSQL/pgvector • Generated 2026-07-15

Table of Contents

1. What Is AuraMatch AI?
.....
2. Visual Tour — The Working Application
.....
3. System Architecture
.....
4. Installation & Setup Guide
.....
5. How the Project Works — User Guide
.....
6. Backend Deep-Dive
.....
7. API Reference & Authentication
.....
8. Testing & Quality Assurance
.....
9. Troubleshooting & Maintenance
.....
10. Further Reading
.....
11. Development Process & AI Tool Usage
.....

New here? Read sections 1–2 for context, then jump straight to **Section 4** to get the app running. Section 5 explains how to actually use it once it's up.

1. What Is AuraMatch AI?

AuraMatch AI is a semantic fragrance matching and recommendation application. It translates natural language context — e.g. *"I need a fresh summer scent for the office commute that projects well"* — into high-dimensional vector embeddings, then runs similarity queries against a database of over **40,000 fragrances** to identify optimal matches and budget-friendly alternatives ("dupes").

1.1 Why This Project Exists

Traditional retail search relies on simple keyword/tag matching, which fails to capture the layered, subjective nature of a fragrance's scent profile. AuraMatch AI instead combines semantic vector search with a **deterministic olfactory decision engine** that evaluates candidates on budget, volatility/longevity, gender leaning, and scent notes — not just nearest-neighbor distance.

1.2 What Makes It Different

Design Choice	Why It Matters
Fully offline core	Embeddings are generated locally with <code>all-MiniLM-L6-v2</code> — no external API is required just to run and search the app.
Fail-safe LLM re-ranking	An optional Groq (Llama 3) layer adds natural-language explanations. A circuit breaker instantly falls back to deterministic results if Groq is slow, down, or unconfigured — zero added latency for the user.
Priority-based ingestion	Duplicate fragrance records are resolved using case-insensitive keys and a source-priority hierarchy, so curated listings win over lower-quality batch imports.
Alembic-managed schema	All schema changes are tracked as migrations, applied automatically on container startup — no manual DB surgery required.
Transparent data quality	Older records without a verified note pyramid are inferred from accords at request time and flagged via <code>has_limited_data</code> — the UI shows a "notes inferred" notice rather than presenting a guess as fact.

1.3 Technology Stack

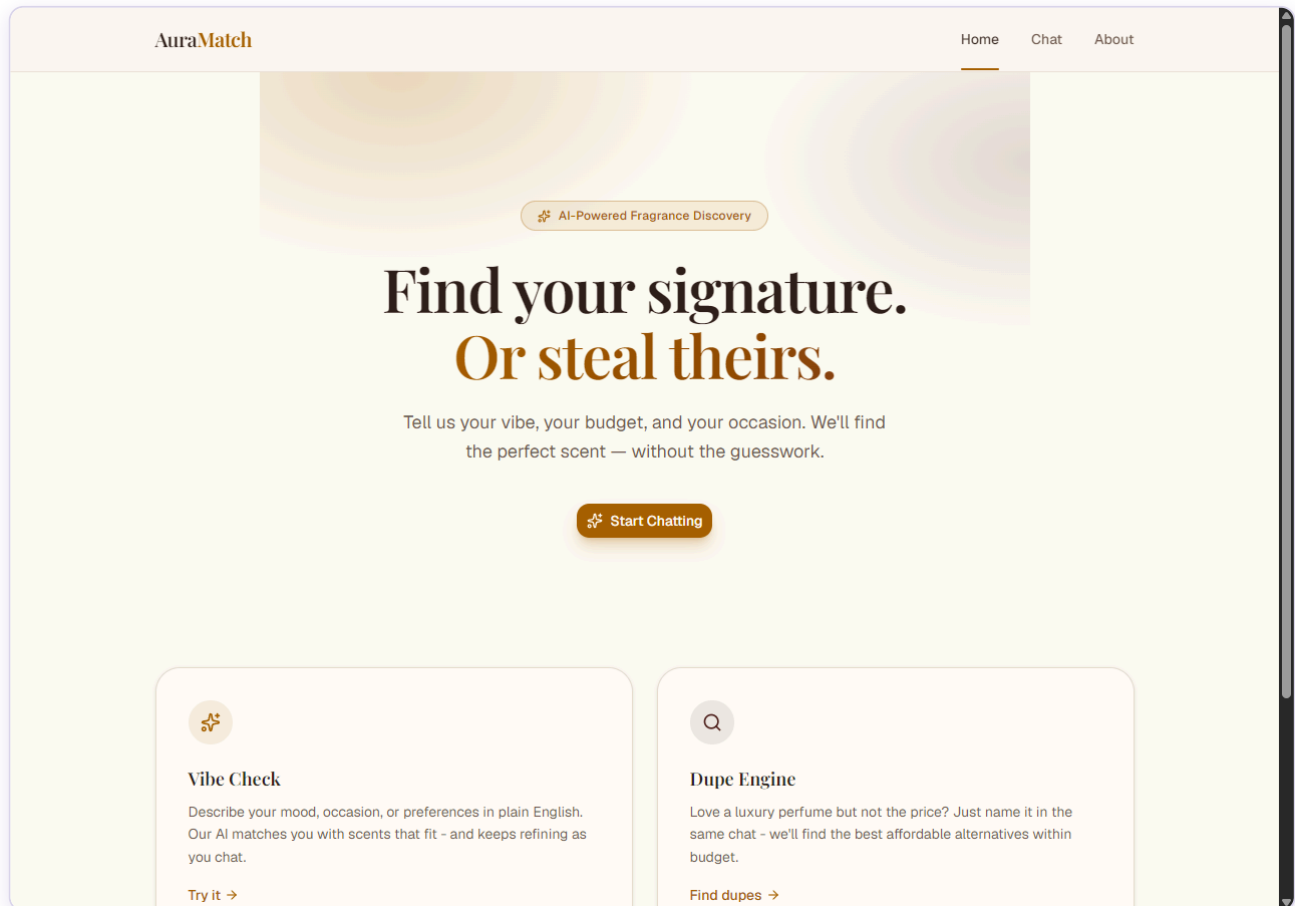
Tier	Technology	Role
Frontend	Next.js 16 (App Router), TypeScript, Tailwind CSS 4, shadcn/ui, Framer Motion	Conversational search UI, perfume detail pages, admin dashboard
Backend	FastAPI (Python 3.11), Pydantic, asyncpg	REST API, auth, rate limiting, scoring engine
Vector Model	<code>all-MiniLM-L6-v2</code> (SentenceTransformers)	384-dimensional local embeddings, no external call needed
Database	PostgreSQL 16 + pgvector + pg_trgm	Vector similarity search, array containment, fuzzy text match
LLM Layer (optional)	Groq Cloud (Llama 3)	Natural-language match explanations, wrapped in a circuit breaker

Migrations	Alembic	Versioned, automatic schema evolution
Orchestration	Docker Compose	Coordinates all three containers on a shared bridge network

2. Visual Tour — The Working Application

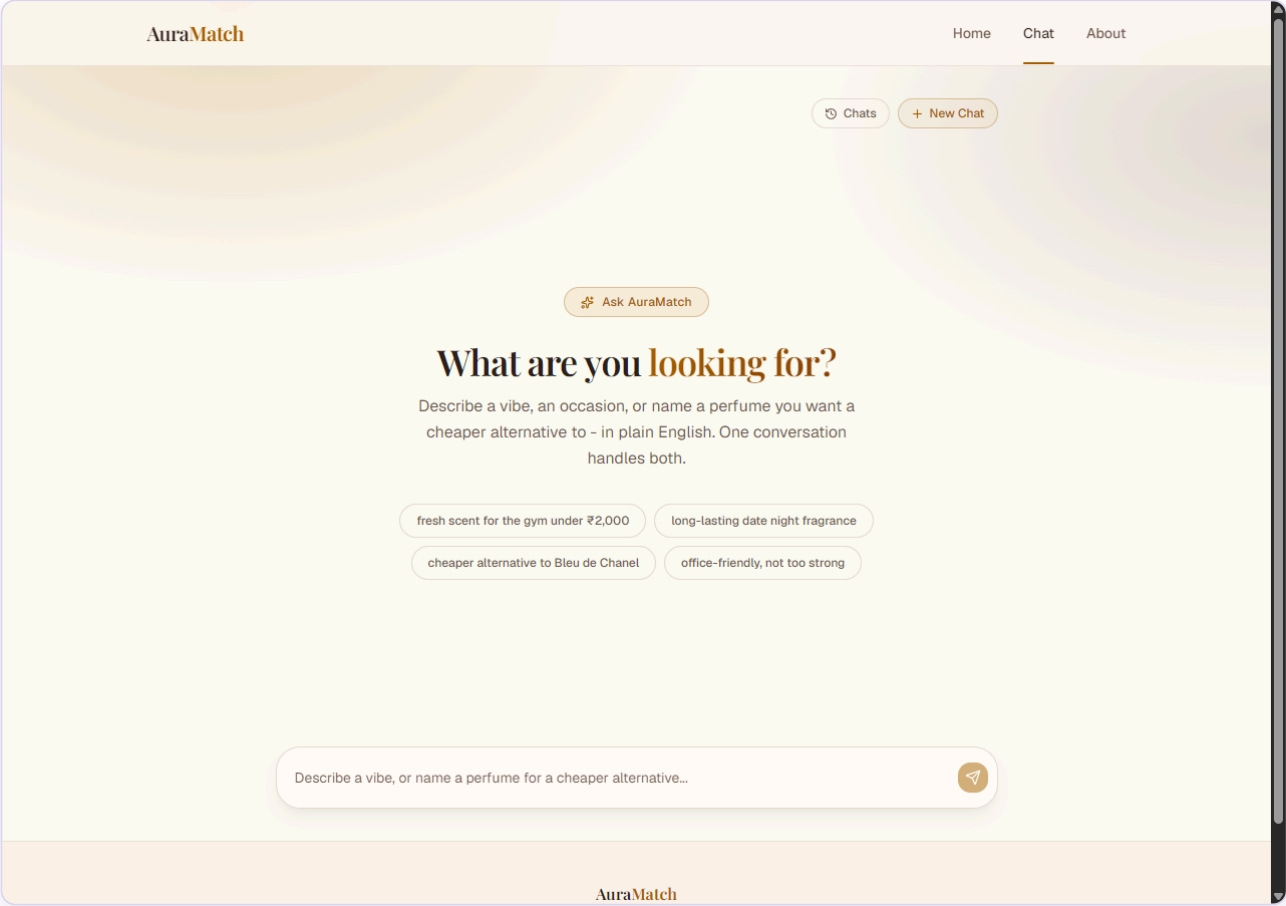
Before diving into setup, here is what you'll actually see once the application is running.

2.1 Landing Page



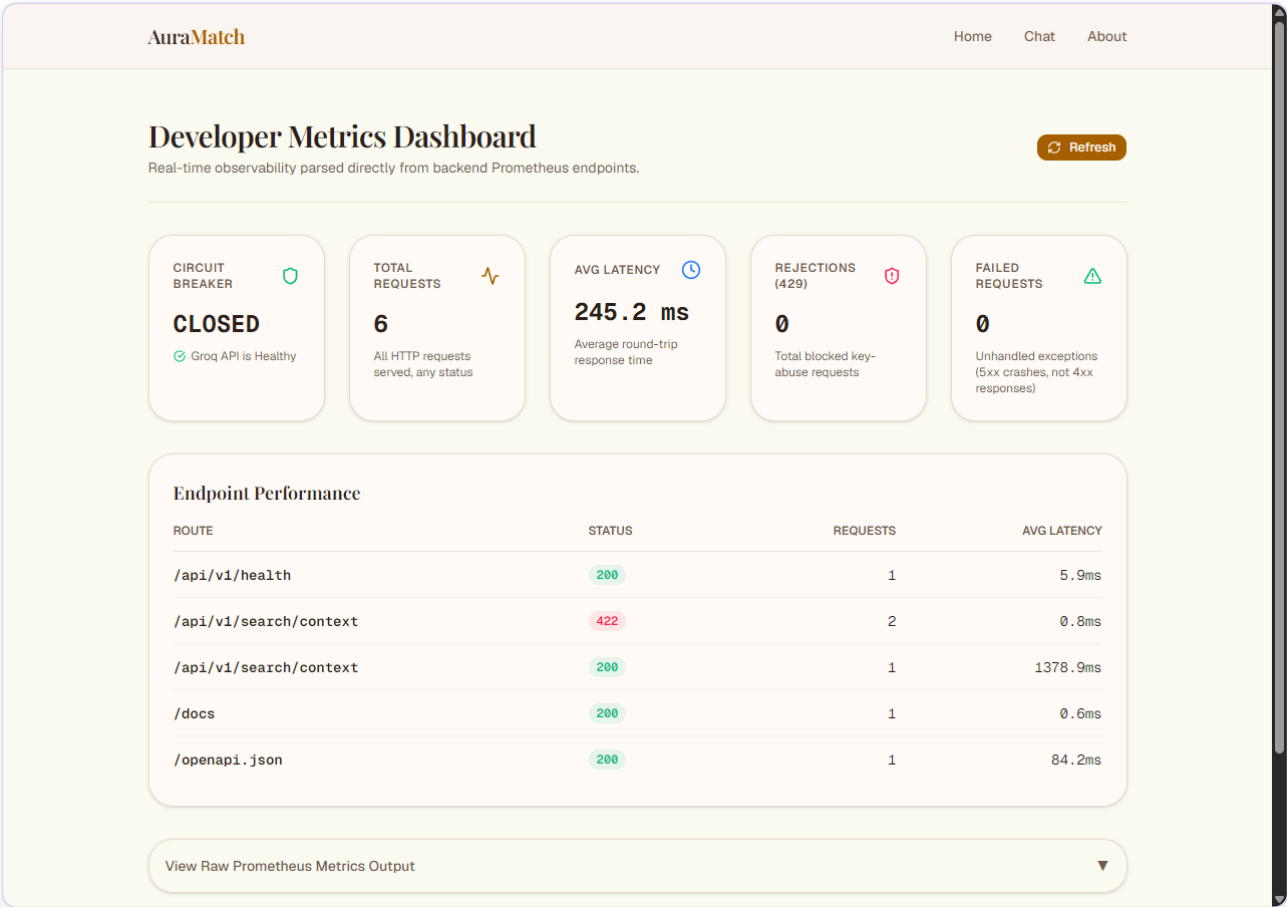
The AuraMatch AI landing / hero page — the entry point at <http://localhost:3000>.

2.2 Vibe Check — Conversational Search



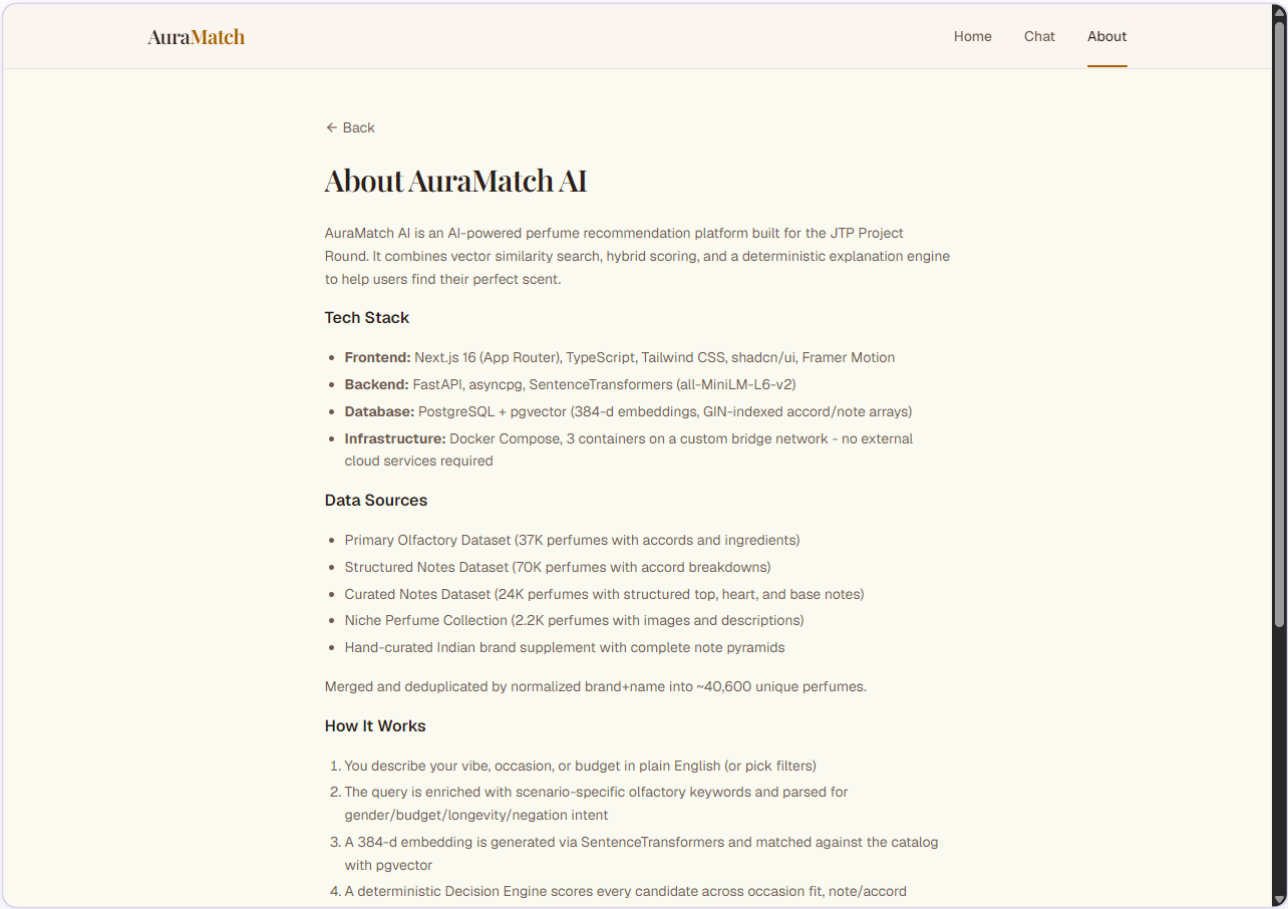
The Vibe Check chat interface at `/search`. Describe a scent in plain English; the system asks clarifying questions and returns ranked matches with note pyramids.

2.3 Developer Metrics Dashboard



The admin/ops dashboard at `/admin` — circuit breaker state, rate-limit rejections, latency, and route breakdown, read live from the Prometheus `/metrics` endpoint.

2.4 About Page



The About page, summarizing the project's purpose and technical approach for visitors.

2.5 Swagger / API Documentation

AuraMatch AI

1.0.0

OAS 3.1

/openapi.json

AI-Powered Fragrance Recommendation Engine

Search

POST

/api/v1/search/context

Context Search

GET

/api/v1/health

Health Check

Dupe Engine

POST

/api/v1/search/dupe

Dupe Search

GET

/api/v1/perfume/{perfume_id}

Get Perfume

default

GET

/

Root

GET

/metrics

Metrics

Schemas

BudgetSearchRequest

>

Expand all

object

ContextSearchRequest

>

Expand all

object

FastAPI's auto-generated interactive API docs at <http://localhost:8000/docs> — every endpoint, schema, and example, testable directly from the browser.

3. System Architecture

AuraMatch AI runs as three containers on a private Docker bridge network (`auramatch_net`): a Next.js frontend, a FastAPI backend, and a PostgreSQL + pgvector database. The optional Groq API is the only component outside this network.

3.1 Network Topology

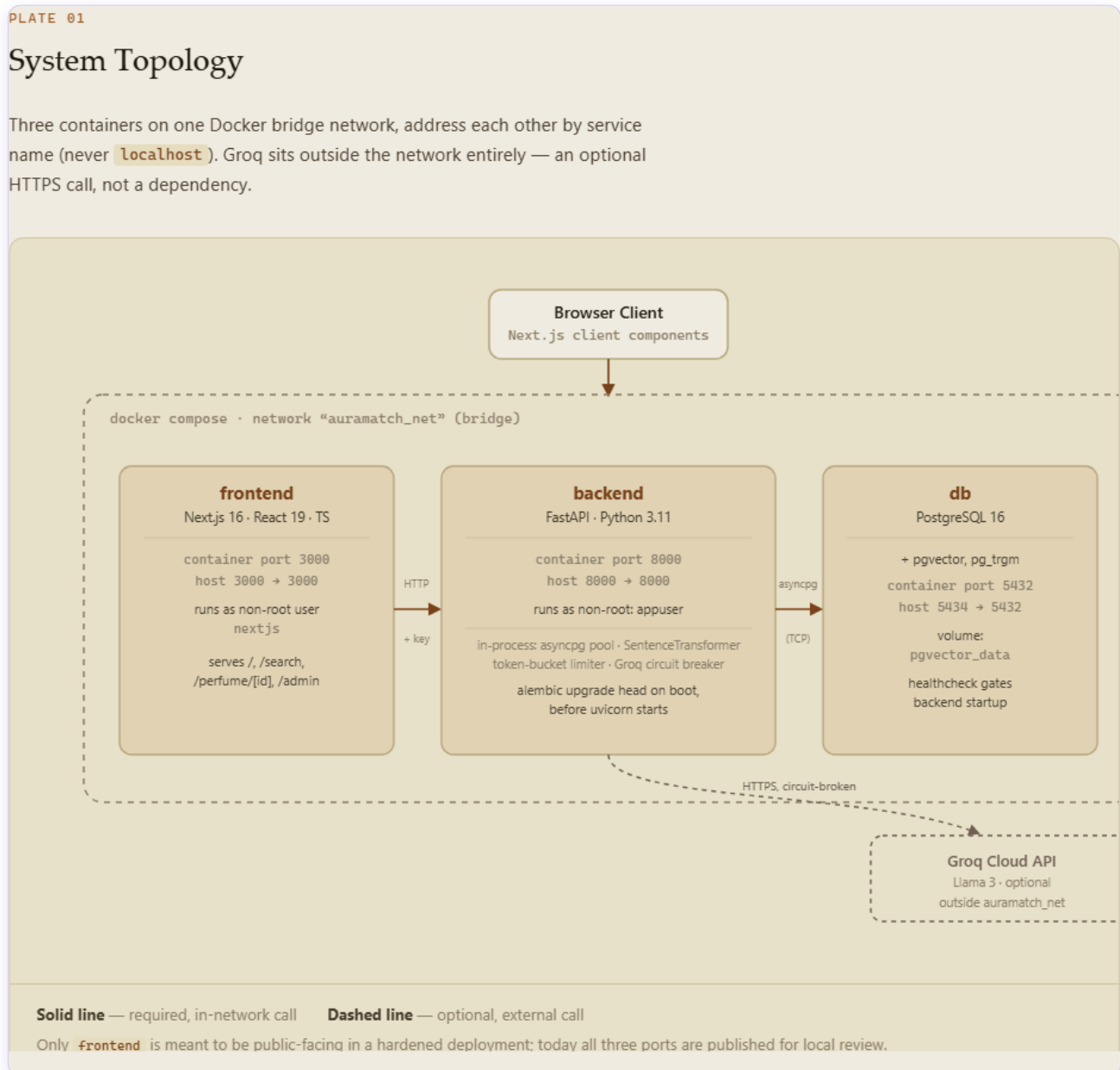


Plate 1 — Three Docker containers on one bridge network; Groq sits outside it as an optional external call.

3.2 Request Lifecycle

Request Lifecycle

One `POST /api/v1/search/context` call, start to finish. Every request passes through auth and rate-limiting before business logic ever runs; the LLM step is the only one that's skippable without changing the response shape.



■ always runs ■ optional / skippable

Step 8 runs on the **full** candidate pool with cheap, numeric-only scoring; step 10 only builds the expensive explanation text for whatever survives truncation. That split exists because the ANN pool width was recently widened 5–20× — explaining every candidate before truncating used to be the slow path.

Plate 2 — The 11-step path a request takes from the browser to a scored response: logging middleware → API-key auth → rate limiter → router → decision engine → database / embedding model → optional Groq re-ranking → response.

3.3 Data Model

PLATE 03

Data Model & Indexes

Two tables carry the whole system: the catalog, and the keys that gate access to it.

Three index types on `perfumes` exist for three unrelated query shapes — nearest-neighbor, array containment, and fuzzy text.

perfumes

~40,600 rows

id	serial, primary key
brand, perfume	text
normalized_key	text — dedup key
main_accords	text[]
notes	text[]
top/heart/base_notes	text[]
price_inr	numeric
gender, type	text
longevity_score	float
sillage_score	float
embedding	vector(384)
source	text
source_priority	smallint
country, perfumer, url	text

derived at read time, not stored:

`has_limited_data` true when notes = [] after cleaning

`top/heart/base_notes`
(fallback) classified from accords when the real columns are still NULL

GIN trigram (pg_trgm) · brand, perfume

index-accelerated word_similarity

typo-tolerant reference-perfume lookup

GIN indexes · notes, main_accords

array containment (&&) in O(log N)

powers negated-notes exclusion pushdown

HNSW index · embedding

vector_cosine_ops · m=16, ef_construction=200

logarithmic ANN search, not a full sequential scan

api_keys

id	serial, primary key
key_hash	sha256, unique
key_type	publishable secret
origin_allowlist	text[]
rate_limit_per_minute	int
revoked_at	timestampz, null

raw key shown once at issuance,

never recoverable after — only the hash persists.

Amber annotations — index, not a column

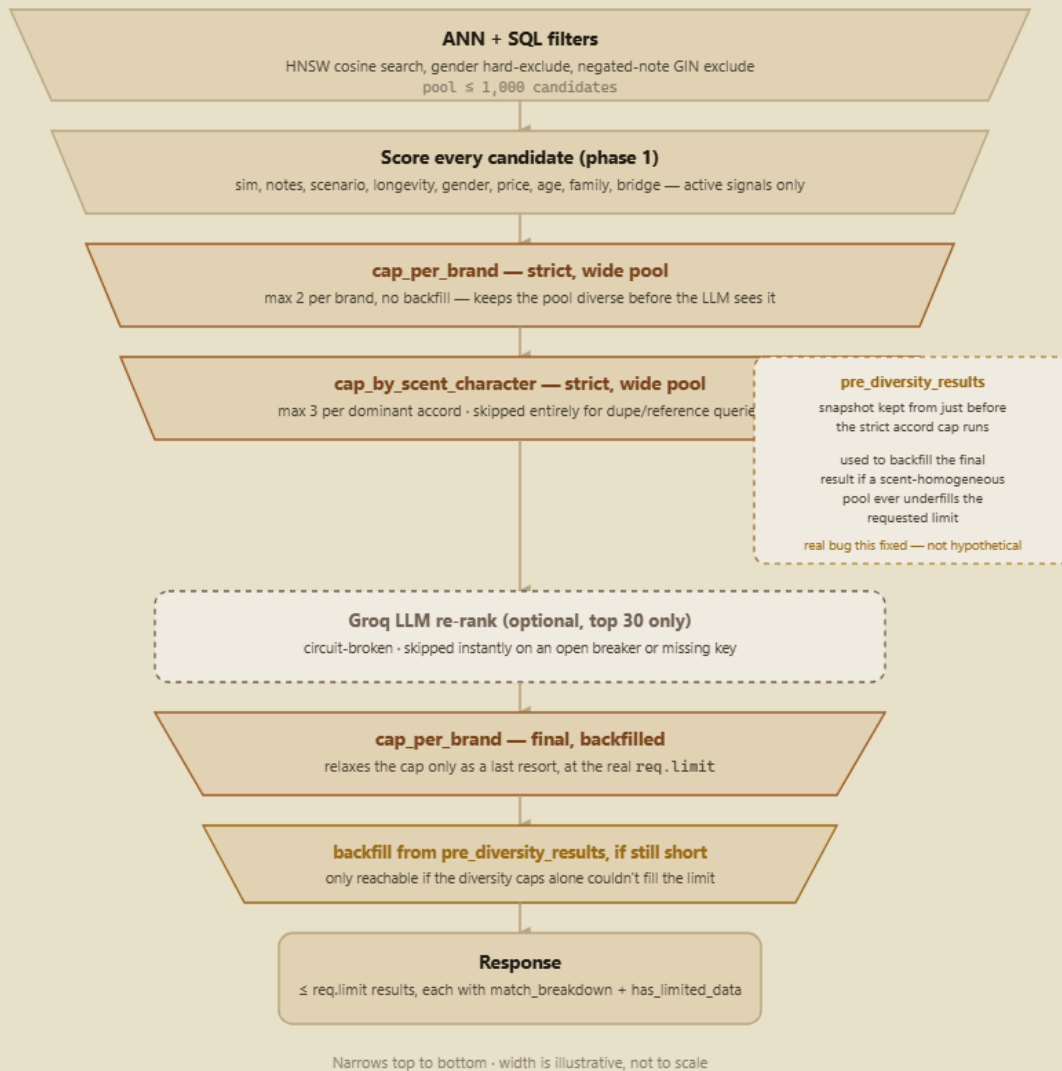
Gold text — computed in Python at read time, never written to disk

Plate 3 — The `perfumes` and `api_keys` tables, with HNSW (vector), GIN (array containment), and trigram (fuzzy text) indexes.

3.4 Scoring & Diversity Pipeline

Scoring & Diversity Pipeline

How a candidate pool of up to a thousand rows narrows to the handful actually shown — and why it narrows in this specific order.



Amber border — a diversity cap (shares one implementation, `_cap_by_key`)

Dashed border — optional / conditional stage

Gold border — a safety net added after a real under-fill bug was found live

The two strict caps (C, D) run **before** the LLM sees anything, deliberately not backfilled at that stage — relaxing the cap to fill a wide pool would let an over-represented brand or accord climb right back up before diversity ever mattered. The final cap (F) is the opposite: backfilled, because a response shorter than requested is a worse outcome than a little repetition.

Plate 4 — How a wide candidate pool from the vector search is narrowed down: scoring, negation filtering, diversity capping, and final ranking.

An interactive, scrollable version of all four diagrams (with full captions) ships with the repo at <documentation/architecture-diagram.html> — open it directly in any browser, no server required.

4. Installation & Setup Guide

tl;dr: `git clone` → `docker compose up --build -d` → open localhost:3000. No API keys required to use the web app.

4.1 Prerequisites

Dependency	Minimum Version	Required For
Docker	20.10.0+	Containerized execution (recommended path)
Docker Compose	2.10.0+	Multi-container orchestration
Git	2.30.0+	Cloning the repository
Python	3.11+	<i>Optional</i> — local (non-Docker) backend dev & tests
Node.js	18.0.0+	<i>Optional</i> — local (non-Docker) frontend dev

4.2 Quick Start (Docker — Recommended)

Step 1 — Clone the repository:

```
git clone https://github.com/shriyashsawant/JTP-PROJECT-ROUND.git
cd JTP-PROJECT-ROUND
```

Step 2 — Build and start every container:

```
docker compose up --build -d
```

This starts three containers — database, backend, frontend — on a shared Docker network. No manual migration or seeding step is required: the database container auto-loads schema + ~40,000 pre-seeded fragrances on first boot, and the backend applies any pending Alembic migrations automatically before it starts serving requests.

Step 3 — Wait for first-boot seeding. First boot restores ~40,000 rows and can take **2–5 minutes** depending on disk speed. Every restart after that is fast — the data persists in a Docker volume (`pgvector_data`).

Verifying the Setup Actually Worked

Don't just guess — confirm all three containers are healthy before opening the app:

```
docker compose ps
```

All three services (`db` , `backend` , `frontend`) should show a status of `Up` (the database as `Up (healthy)`). If backend or frontend show `Restarting` , the database probably hasn't finished seeding yet — give it another minute and check again.

Then confirm the backend can actually reach the database:

```
curl http://localhost:8000/api/v1/health
```

Expected response: `{"status":"ok","db_connected":true}`. If you don't have `curl`, just open that URL in a browser.

To watch what the backend is doing during seeding/startup (e.g. confirm migrations applied):

```
docker compose logs backend
```

4.3 Accessing the Application

Interface	URL	Notes
Web UI	localhost:3000	Vibe Check chat, Dupe Finder, fragrance detail cards. No API key needed — a publishable key is pre-baked into the frontend build.
Swagger API Docs	localhost:8000/docs	Interactive endpoint reference. To actually execute a search here via "Try it out" you need an API key — see §4.5.
Admin Dashboard	localhost:3000/admin	Live circuit-breaker state, rate-limit stats, latency.
Prometheus Metrics	localhost:8000/metrics	Raw scrape target backing the admin dashboard. Unauthenticated, same convention as <code>/health</code> .

4.4 Configuring Groq LLM Re-Ranking (Optional)

The app runs fully offline by default. To additionally enable AI-generated natural-language explanations for each match:

1. Make sure you're in the project root (the folder containing `docker-compose.yml`, `backend/`, and `frontend/`).
2. Create a file named exactly `.env` (leading dot, no extension) at that root.
3. Add your key, obtained from the [Groq Console](#) (API Keys section → Create API Key):

```
GROQ_API_KEY=gsk_your_actual_groq_api_key_here
```

4. Restart the containers so Compose picks up the new variable:

```
docker compose up -d
```

If no key is set, or Groq is unreachable, the app falls back instantly to deterministic matching — there is no degraded or broken experience for the end user.

Verifying it's active — any one of:

- Tail `docker compose logs backend` and look for outbound prompt/response logs when you run a search.
- Open the [admin dashboard](#) and check the Circuit Breaker section: `CLOSED` = healthy and routing to Groq, `OPEN` = 5 consecutive failures, auto-degraded.

- Query `/metrics` for the gauge `auramatch_circuit_breaker_state{breaker="groq"}` — `0` means active/healthy.

4.5 Issuing API Keys (for Swagger / Postman / curl)

The web UI already has a key baked in and needs nothing extra. To call the API *directly* (e.g. through Swagger's "Try it out", curl, or Postman), issue yourself a secret key:

```
docker compose exec backend python scripts/issue_api_key.py --type secret --label "manual testing" --rate-limit 300
```

Or, if running the backend outside Docker:

```
cd backend
python scripts/issue_api_key.py --type secret --label "manual testing" --rate-limit 300
```

Important: the raw key is printed exactly once at creation time and cannot be recovered afterward — copy it somewhere safe immediately. Send it as an `X-API-Key` header on every request.

4.6 Local Developer Setup (Non-Containerized)

Prefer running components directly on your host for development or debugging?

4.6.1 Database

1. Install PostgreSQL 16 and the `pgvector` extension.
2. Create a database named `auramatch` and a user with write privileges.
3. Run `backend/data/01_schema.sql` to initialize the tables.

4.6.2 Backend

```
cd backend
python -m venv .venv
```

Activate the virtual environment:

```
# Windows (CMD/PowerShell):
.venv\Scripts\activate
# Linux/macOS:
source .venv/bin/activate
```

Install dependencies:

```
pip install -r requirements.txt
pip install -r requirements-dev.txt
```

Copy `.env.example` to `.env` and set your database URL:

```
DATABASE_URL=postgresql://<user>:<password>@localhost:5432/auramatch
```

Apply migrations and start the server:

```
alembic upgrade head
uvicorn app.main:app --reload --port 8000
```

4.6.3 Frontend

```
cd ../frontend
npm install
```

Create a `.env.local` file:

```
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_API_KEY=pk_live_f3o8eLGSmio2iWVDzyG25ppqPVz0sCMY_5vUh6cAtZ8
```

This is the same publishable key already baked into the Dockerized frontend build — it is safe to embed in browser JS by design (see §7.1), restricted only by an `Origin` allowlist, not a real secret.

```
npm run dev
```

4.7 Stopping, Updating & Resetting

Action	Command
Stop everything (keep data)	<code>docker compose down</code>
Pull latest code and rebuild	<code>git pull && docker compose up --build -d</code>
Tail logs for a service	<code>docker compose logs -f backend (or frontend , db)</code>
Full reset (wipes seeded data!)	<code>docker compose down -v && docker compose up --build -d</code>

`docker compose down -v` permanently deletes the seeded database volume. Only use it when you actually intend to start over from a clean 40K-row seed.

5. How the Project Works — User Guide

Open <http://localhost:3000>. The landing page is the entry point into the discovery engine.

5.1 Vibe Check — Conversational Matching

Click **Start Chatting**, or use the **Chat** tab in the navbar. Describe a scent in plain English:

- "I want a fresh, citrusy perfume for office commute that lasts all day"
- "A dark, cozy woody fragrance for winter nights"

The system tracks conversation history, so you can refine with follow-ups at any point:

- "Can you make it more masculine?"
- "Show me only options under ₹4,000."
- "Add some leather or tobacco notes."

Sliders and dropdowns in the search bar/sidebar let you directly adjust:

Control	Purpose
Budget Limit	Maximum price in INR (₹)
Gender Bias	Unisex, leaning masculine, leaning feminine
Concentration Level	EDT, EDP, Extrait, Cologne, etc.
Longevity & Sillage Weights	Prioritize scents that last longer or project further

5.2 Dupe Engine — Budget Alternatives

Want an affordable alternative to a luxury perfume? Ask directly in the same chat interface:

- "Show me affordable alternatives to Creed Aventus under ₹5000"
- "Suggest a dupe for Chanel No. 5"

The system looks up the referenced luxury perfume, extracts its exact scent profile (accords + notes pyramid), then runs a nearest-neighbor vector search to surface similar-smelling matches at a fraction of the price.

5.3 Reading a Scent Detail Card

Element	Description
Fragrance Name & Brand	The designer or perfume house
Price (INR)	Calculated using standardized brand-tier pricing rules
Main Accords	Badges for dominant scent notes (Citrus, Woody, Vanilla, Leather, ...)
Longevity & Sillage	Derived from chemical note volatility. Longevity = how long it lasts on skin (e.g. "8.5 / 10 hours"); Sillage = how far it projects (e.g. "Strong", "Intimate")

Notes Pyramid	Top notes (immediate, volatile citruses/herbs) → Heart notes (the main character: flowers, fruits, spices) → Base notes (the dry-down: musks, woods, resins, amber)
AI-Generated Explanation	If Groq re-ranking is enabled, a short natural-language "why this matches" explanation

5.4 Developer Metrics & Admin Dashboard

Available at </admin>, reading live from the backend's Prometheus `/metrics` endpoint:

Metric	What It Shows
Circuit Breaker State	Groq connection status — <code>CLOSED</code> (healthy), <code>OPEN</code> (failed over to offline mode), <code>HALF-OPEN</code> (testing recovery)
Rate Limit Rejections	Requests rejected by the token-bucket limiter
Average Latency	Average response time in milliseconds
Route Breakdown	Request counts and status codes per API route

5.5 Sample Queries to Try

Query Type	Example Input	What the Engine Evaluates
Simple Vibe	"Fresh clean laundry smell"	Light aldehydes, musks, laundry-clean accords
Occasion-Based	"Seductive date night fragrance"	Amber, vanilla, spices, heavier base notes, higher sillage
Negative Search	"Fresh woody scent without any rose or patchouli"	Detects "without" → applies a negative-penalty filter removing rose/patchouli candidates
Weather/Season	"Crisp summer scent"	Boosts citrus, marine notes, mint; filters out heavy winter notes
Dupe Request	"Dupes for Tom Ford Tobacco Vanille under ₹3000"	Matches Tom Ford's profile (tobacco, cacao, spices, vanilla) within the budget cap

6. Backend Deep-Dive

For readers who want to know what's actually running under the hood, not just how to start it.

6.1 Request Path

Every request to a product route (search, dupe, perfume detail) passes through, in order: logging middleware → API-key auth dependency → token-bucket rate limiter → router → decision engine → database / local embedding model → (optionally) Groq re-ranking behind a circuit breaker → response. `GET /api/v1/health` is the one route that skips auth entirely, per standard liveness-probe convention.

6.2 Key Backend Modules

File	Responsibility
<code>app/main.py</code>	Entry point; inline request-logging middleware; CORS allowlist; lifespan hook that warms the DB pool + embedding model at boot
<code>app/core/config.py</code>	Pydantic-settings config: DB URL, model name, CORS origins, optional Groq key, feature flags
<code>app/api/routes_search.py</code>	<code>POST /search/context</code> , <code>GET /health</code>
<code>app/api/routes_dupe.py</code>	<code>POST /search/dupe</code> , <code>GET /perfume/{id}</code>
<code>app/api/auth.py</code>	<code>require_api_key</code> dependency: validates hashed key, Origin allowlist, per-key rate limit
<code>app/services/db_repository.py</code>	Raw SQL execution, connection pooling, vector parsing, GIN array pushdowns, accord-based pyramid fallback
<code>app/services/decision_engine.py</code>	Core matching algorithm: scoring, scenario mapping, chemical bridge checks, gender/concentration modifiers
<code>app/services/intent_detector.py</code>	Deterministic regex/keyword parser extracting longevity, budget, sillage, gender, negation intent
<code>app/services/ml_engine.py</code>	Loads the local SentenceTransformer model; runs inference in a thread pool so it never blocks the event loop
<code>app/services/circuit_breaker.py</code>	Generic async CLOSED/OPEN/HALF_OPEN breaker; trips after 5 consecutive Groq failures
<code>app/services/rate_limiter.py</code>	In-memory token-bucket limiter, keyed per API key (+ client IP for publishable keys)
<code>app/services/llm_enrichment.py</code>	Builds prompts and calls Groq for explanations + checklist validation
<code>app/db/migrations/</code>	Alembic migration history (0001→0004); the sole source of truth for schema changes, applied automatically on container start

6.3 Database Indexing

Index Type	Applied To	Why
------------	------------	-----

HNSW (<code>vector_cosine_ops</code> , M=16, ef_construction=200)	<code>embedding</code> column	Logarithmic-time approximate nearest-neighbor search; flat indexes fall back to sequential scans past ~10K rows
GIN	<code>notes</code> , <code>main_accords</code> arrays	O(log N) array-containment queries (<code>&&</code>), used for SQL-level negation pushdown
GIN Trigram (<code>pg_trgm</code>)	<code>brand</code> , <code>perfume</code> text columns	Fast, index-accelerated fuzzy/case-insensitive matching for the Dupe Engine's "find the parent perfume" step

6.4 Frontend Pages

Route	Purpose
<code>/</code>	Landing page — hero and entry point into <code>/search</code>
<code>/search</code>	The single search surface: a conversational chat handling both open-ended "vibe" queries and dupe/reference queries through one endpoint; persists every conversation to <code>localStorage</code>
<code>/perfume/[id]</code>	Detail view: metadata, launch year, country, scent pyramid (flags inferred pyramids), and a "Find Dupes" action
<code>/admin</code>	Ops metrics dashboard parsing <code>/metrics</code> client-side; intentionally not linked from the public nav

7. API Reference & Authentication

7.1 Endpoints

Method	Endpoint	Description
POST	<code>/api/v1/search/context</code>	Evaluates natural language + explicit filters, returns ranked matches
POST	<code>/api/v1/search/dupe</code>	Finds affordable alternatives within a budget limit
GET	<code>/api/v1/perfume/{id}</code>	Metadata,illage/longevity scores, and note pyramid for one perfume
GET	<code>/api/v1/health</code>	Database connectivity check; the only unauthenticated product route
GET	<code>/metrics</code>	Prometheus scrape target: latency, error rates, DB pool, circuit-breaker, rate-limit rejections

7.2 Two-Tier API Key Model

The frontend calls the API directly from browser JS (no server-side proxy), so a plain secret key can't be used there — anyone could read it via devtools. AuraMatch AI solves this the same way Stripe, Google Maps, and Mapbox do: two key types.

	<code>publishable</code>	<code>secret</code>
Safe to embed in browser JS?	Yes	Never
Restricted by	<code>Origin</code> header allowlist	Nothing (server callers don't reliably send <code>Origin</code>)
Intended caller	The AuraMatch web frontend, or any first-party browser client	A genuine third-party server-to-server integration
Rate-limit bucket	<code>(key, client_ip)</code>	<code>(key,)</code>

Only `sha256(raw_key)` is ever persisted. The raw key is shown exactly once at issuance.

Issuing a key:

```
# Publishable (frontend-safe), restricted to specific origins:
python backend/scripts/issue_api_key.py --type publishable \
  --label "My Web App" --origin https://app.example.com

# Secret (server-to-server), no origin restriction:
python backend/scripts/issue_api_key.py --type secret \
  --label "My Integration" --rate-limit 300
```

Calling the API:

```
curl -X POST http://localhost:8000/api/v1/search/context \
  -H "X-API-Key: sk_live_..." \
```

```
-H "Content-Type: application/json" \  
-d '{"query": "fresh citrus scent for the gym", "limit": 5}'
```

7.3 Error Responses

Status	Meaning
401	Missing <code>X-API-Key</code> header, or the key is unknown/revoked
403	A <code>publishable</code> key was used from a non-allowlisted <code>Origin</code>
429	Rate limit exceeded for this key (and client IP, for publishable keys)

All error bodies follow `{"detail": "<message>"}`.

7.4 Rate Limiting

Each key has its own `rate_limit_per_minute` (default 60), enforced as a token bucket: a full minute's requests are allowed as a burst, refilling continuously afterward rather than resetting hard every 60 seconds. This is in-memory, per backend process — correct for the current single-process deployment.

7.5 Versioning

The current API is `/api/v1` and is considered stable: existing endpoints, fields, and response shapes will not change in a breaking way under this prefix. Breaking changes ship as `/api/v2`, with `/api/v1` continuing to work until explicitly deprecated.

8. Testing & Quality Assurance

The application is validated by a suite of **287 unit and integration tests** covering matching logic, intent detection, circuit-breaker/rate-limiter behavior, API key authentication, schema validation, database fallbacks, and observability metrics.

Run the suite locally:

```
cd backend  
.venv\Scripts\python -m pytest
```

Mypy type checking and Ruff lint configurations are fully clean across all core service files.

9. Troubleshooting & Maintenance

Symptom	Cause & Fix
Port 3000 or 8000 already in use	Another local process is bound to that port. Remap the host port in <code>docker-compose.yml</code> (e.g. <code>"3001:80"</code> for the frontend).
Database connection fails	The database container may still be initializing or seeding. Tail its logs: <code>docker compose logs db</code>
First query feels slow	The <code>all-MiniLM-L6-v2</code> transformer is loading into memory — a one-time ~2–3 second delay. Every query after that is fast.
Need to reset the database from scratch	<code>docker compose down -v</code> then <code>docker compose up --build -d</code> — drops the volume and re-seeds clean.
Containers show "Restarting"	Usually the database hasn't finished seeding yet. Wait a minute, then re-check with <code>docker compose ps</code> .
Groq explanations never appear	Confirm <code>.env</code> exists at the project root (not inside <code>backend/</code>), contains a valid <code>gsk_...</code> key, and that you ran <code>docker compose up -d</code> again after creating it.

9.1 Environment Files — What Goes Where

File	Location	Holds
<code>.env</code>	Project root	<code>GROQ_API_KEY</code> only — read by Docker Compose and passed into the backend container
<code>.env</code>	<code>backend/</code>	<code>DATABASE_URL</code> and other backend settings, for non-Docker local dev only
<code>.env.local</code>	<code>frontend/</code>	<code>NEXT_PUBLIC_API_URL</code> + <code>NEXT_PUBLIC_API_KEY</code> (the publishable key), for non-Docker local dev only

10. Further Reading

This guide consolidates the project's full documentation set for a first-time reader. Each topic has a deeper, engineering-level reference in the repository under [documentation/](#) :

- **SYSTEM_ARCHITECTURE.md** — full component diagram, file-by-file breakdown, deployment topology
- **DECISION_ENGINE.md** — the olfactory scoring algorithm, scenario maps, negation logic, gender/concentration modifiers
- **DATA_INGESTION_PIPELINE.md** — Alembic migrations, validation contracts, source-priority deduplication
- **FRONTEND_ARCHITECTURE.md** — page-by-page walkthrough, conversational flow, state management
- **TESTING_AND_OBSERVABILITY.md** — test coverage, mocks/doubles, logging middleware, metrics roadmap
- **THIRD_PARTY_API.md** — the full API key model and integration guide
- **GROQ_SETUP.md** — obtaining and verifying a Groq API key
- **architecture-diagram.html** — interactive version of the four architecture plates in Section 3

AuraMatch AI — Complete Project Guide · Compiled from the project's documentation set for new-user onboarding.

11. Development Process & AI Tool Usage

This project was built with substantial use of AI tools across the full development lifecycle — planning, implementation, task tracking, and documentation. Different tools were used for different stages rather than one tool for everything:

Tool	Role in This Project	Why It Was Used
Claude Code	Multi-agent AI pair programming for the bulk of the lifecycle — architecture decisions, implementation, test writing, and this documentation set itself	Handles large multi-file changes and longer-running agentic work (parallel subagents) rather than single-turn chat
Gemini (and other LLMs)	Drafting PRDs and early technical/product planning before implementation began	Used for broad ideation/planning passes ahead of the more execution-focused coding work
Antigravity	Task and sprint planning — breaking the plan into ordered, actionable implementation tasks	Kept work scoped and sequenced so coding sessions had a clear task queue instead of ad hoc requests
Obsidian	Persistent memory — cross-session notes capturing decisions, rejected approaches, and context that would otherwise be lost between sessions	Gave continuity across sessions/tools so prior decisions didn't need to be re-derived or re-explained each time

Every AI-assisted change in this codebase followed the same discipline: a claim or plan proposed by an AI tool was verified against the actual running system (live database queries, real HTTP requests against the containers, full test/lint/type-check runs) before being accepted, not taken on faith. Several early AI-generated architectural proposals in this project's history were explicitly rejected or scaled back — e.g. a generic "enterprise blueprint" (hexagonal architecture, Redis caching, a full OpenTelemetry mesh) was assessed as disproportionate for this system's actual scale, and only its one genuinely justified piece was implemented — precisely because that verify-before-trust discipline was applied throughout, not skipped for convenience. The author remains able to explain and justify every decision and every line of code in this repository, per the project's own AI-tool usage policy.